

Topic 4 Review [99 marks]

1. [Maximum mark: 6]

25M.1.SL.TZ2.5

Construct a trace table for the following algorithm:

```
N = 5
S = 0
R = 0
loop while N > 0
  A = N mod 3
  if A = 0
    then
      S = S - N
    else
      if A = 1
        then
          S = S + N
        else
          S = S + 1
      end if
    end if
    R = R + S
    N = N - 1
  end loop
output ('The result is ', R)
```

[6]

Markscheme

*Award **max** [6].*

Award [1] for a trace table with at least 4 columns, excluding the output column.

Award [1] for each correct column (column heading: N, S, R, A, output, see the example)

Note: *The trace table may be differently presented.*

N	S	R	A	output
5	0	0	2	
4	1	1	1	
3	5	6	0	
2	2	8	2	
1	3	11	1	
0	4	15		The result is 15

Examiners report

Many students provided correct answers here. Most were able to earn at least three marks for drawing the table with the correct columns and having at least two columns accurate. However, some only included the value of R in the output column instead of the full phrase.

2. [Maximum mark: 15]

25M.1.SL.TZ2.11

There are 200 students in a school. Their names are held in the one-dimensional string array **STUDENTS**.

The one-dimensional integer array **MARKS** stores marks (0–100 inclusive) that students scored in an examination.

Figure 1: Example data stored in the two arrays **STUDENTS and **MARKS****

STUDENTS		MARKS	
[0]	Ximena Alba	[0]	5
[1]	Boris Mount	[1]	88
[2]	Hugh Parr	[2]	45
[3]	Arjinder Singh	[3]	75
[4]	Mei Chen	[4]	59
	...		...
[199]	Tammy Teller	[199]	51

The one-dimensional integer array **GRADES** will be used to store the grades awarded to students based on their examination marks.

In **Figure 1**, Boris Mount scored 88 marks. His grade will be stored in **GRADES[1]**.

Consider the following algorithm that the school currently uses for awarding grades:

```

loop K from 0 to 199
  GRADES[K] = 1 + (MARKS[K] div 10)
  if GRADES[K] >= 7 then
    GRADES[K] = 7
  end if
end loop

```

(a.i) Determine the value of **GRADES[0]**.

[1]

Markscheme

*Award **max**[1].*

1;

Examiners report

Many correct answers were given here.

(a.ii) Determine the value of GRADES[1].

[1]

Markscheme

*Award **max** [1].*

7;

Examiners report

This question was generally answered well.

(a.iii) Determine the value of GRADES[2].

[1]

Markscheme

*Award **max** [1].*

5;

Examiners report

Few students failed to appropriately understand the use of DIV and gave answer as a decimal number.

(a.iv) State the minimum mark necessary to achieve Grade 7.

[1]

Markscheme

*Award **max**[1].*

60;

Examiners report

Many correct answers were given here.

Grade 1 is a failing grade. Each student who receives a failing grade must re-sit the examination.

- (b) Construct an algorithm in pseudocode to fill the one-dimensional string array **RESIT** with the names of students who must re-sit the examination.

[4]

Markscheme

*Award **max**[4].*

Award [1] for a correct loop;

Award [1] for checking if a failing grade;

Award [1] correct indexing used to access values in at least two arrays;

*Award [1] for initializing and incrementing (if needed) the index (**i**) in the array **RESIT**;*

*Award [1] for correctly assigning the name from the array **STUDENTS** to the array **RESIT**;*

Example 1 (uses the **MARKS** array):

```
// assume: the array RESIT of sufficient size is initialized
```

```
// the minimum mark for grade 2 (passing grade) is 10
```

```
i = 0
```

```
loop K from 0 to 199 //Accept len(STUDENTS)–1 or STUDENTS.length()
```

```
    //accept any correct loop through the array MARKS
```

```

    if MARKS[K] < 10
    then
        RESIT[I] = STUDENTS[K]// Accept RESIT.append(STUDENTS[K])
        I = I + 1
    end if
end loop

```

Example 2 (uses the GRADES array defined in part(a)):

```

// assume: the array RESIT of sufficient size is initialized
// grade 1 is a failing grade
I = -1
loop K from 0 to 199 //Accept len(STUDENTS)-1
    //accept any correct loop through the array GRADES
    if GRADES[K] = 1
    then
        I = I + 1
        RESIT[I] = STUDENTS[K]// Accept RESIT.append(STUDENTS[K])
    end if
end loop

```

Note: Collection methods not accepted.

Examiners report

Many students were able to answer this question successfully scoring full marks. Students who used add item or add method to the array lost a mark.

A different method of awarding grades is proposed for the examinations. This new grading system will use three grades represented by the letters A, B, and C.

Grades A, B, or C will be calculated as follows:

- The average mark for all students is calculated.
- Grade A is awarded if an individual student's marks are more than 20 marks above the average mark for all students.

- Grade B is awarded if an individual student's marks are within 20 marks of the average mark for all students.
- Grade C is awarded if an individual student's marks are more than 20 marks below the average mark for all students.

For example, if the average mark for all students is 49.5:

- Grade A is awarded if a student's marks are greater than 69.5.
 - Grade B is awarded if a student's marks are in the range from 29.5 to 69.5 inclusive.
 - Grade C is awarded if a student's marks are less than 29.5.
- (c) Construct an algorithm in pseudocode to calculate and store the letter grades of all students in the one-dimensional string array **LETTERGRADES** as described.

[7]

Markscheme

*Award **max** [7].*

*Award **max** [3] for the calculation of the average mark. [1] for initializing sum and adding Marks [k] to the sum, [1] for a correct loop, [1] for the sum divided by 200;*

Award [1] for a correct second loop;

*Award [1] for correct use of indexing in both **MARKS** and **LETTERGRADES** array;*

*Award **max** [3] for determining each letter grade. [1] for each condition and grade assignment (String) to the **LETTERGRADES** arrayx3;*

//calculate the average mark

Sum=0

loop K from 0 to 199 //Accept len(STUDENTS)–1 or

STUDENTS.length()–1

Sum = Sum + MARKS[K] // Accept any correct loop

end loop

AVE = Sum / 200 //accept SUM DIV 200

K=0

```

loop K from 0 to 199 //Accept len(STUDENTS)–1 or
STUDENTS.length()–1
    // loop through the array MARKS
    // accept any correct loop
    if MARKS[K] > AVE + 20    //determine the grade
    then
        G = "A"
    else
        if MARKS[K] < AVE – 20
        then
            G = "C"
        else
            G = "B"
        end if
    end if
    LETTERGRADES[K] = G
end loop

```

Note: Collection methods not accepted.

Alternate selection constructs accepted provided it leads to the correct grade assignment

Examiners report

A good number of students responded well to this question with majority of them gaining at least 3 to 4 marks. Some common mistakes was the use of Grade array to compare the average instead of the Marks array. Assignment of letter grade example 'A' is a string and should be within quotes. Again, using add or add item to add to the LetterGrades array led to another loss of mark. Some students obtained average by dividing with 199 instead of 200.

3. [Maximum mark: 15]

25M.1.HL.TZ2.13

A team of four high-school students decided to create a computer program for younger students to help them learn to count and recognize whole numbers from 1 to 25.

A table partially completed with numbers from 1 to 25 is given to a younger student. The table consists of five rows and five columns (see **Figure 2**). The younger student must enter the remaining numbers. Each number from 1 to 25 should be entered only once.

Figure 2: Example table partially filled with numbers

	24	1		
		7	14	
4	6			22
10			21	3
11		25		9

Each of the four students is responsible for creating several sub-programs that will be included in the program.

(a) List **three** advantages of using sub-programs in this situation.

[3]

Markscheme

*Award **max** [3].*

The (difficult) program is divided into several easier parts (sub-programs);
The program is better organized/easier to understand/reduced size of code
(due to several sub-programs calls)/no duplication of code;
Sub-programs hide implementation details;

Sub-programs keep specific tasks self-contained;
Each of the sub-programs could be written independently;
Each of the sub-programs is debugged/tested independently;
Different parts of the program can be written at the same time/concurrently;
Sub-programs speed up the development process (as different students can work on different subprogram at the same time);
Each sub-program could be used many times in this program (and in other programs);
The maintenance/modification/change of the program will be easier/faster (because only a sub-program could be changed/modified as needed);

Examiners report

Most students effectively listed the advantages of using subprograms and were able to achieve full marks.

The sub-program `inputInRange ()` should return a valid input number.

This will ensure that the younger student's input is a number in the range 1–25 inclusive. If the input number is not in this range, an appropriate message is output. The process will be repeated until a valid number is input.

For example:

For the sub-program call `N = inputInRange ()`, if the following appears on the screen:

Enter a number:

 The number is not in the range from 1 to 25. Please try again.
 Enter a number:

 The number is not in the range from 1 to 25. Please try again.
 Enter a number:

 Your number is valid.

Then the value of **N** would be 15 after execution.

- (b) Construct an algorithm in pseudocode for the sub-program `inputInRange()`.

[4]

Markscheme

*Award **max** [4].*

Award [1] for a correct while loop

Award [1] for a correct condition/validation check

Award [1] for input within the while loop (input repeated until input validated)

Award [1] for outputting appropriate messages

Award [1] for return statement;

Example 1:

`inputInRange()`

`K=input('Enter a number: ')`

`loop while (K < 1) or (K > 25)`

`output('The number is not in the range 1 to 25. ')`

`output(' Please try again! ')`

`K=input('Enter a number: ')`

`end loop`

```
    output('Your number is valid. ')
    return K
end inputInRange()
```

Example 2:

```
inputInRange()
    F=true
    loop while F
        output('Enter a number')
        input(K)
        if(K < 1) or (K > 25)
            then
                output('The number is invalid. ')
                output( 'Please try again! ')
            else
                F=false
            end if
        end loop
        output('Your number is valid. ')
        return K
    end inputInRange()
```

Example 3:

```
inputInRange()

    loop while True
        K=input('Enter a number: ')
        if(K >= 1) and (K <= 25)
            then
                output('Your number is valid. ')
                return K
            else
                output('The number is not in the range1 to 25. ')
                output('Please try again! ')
            end if
        end loop
    end inputInRange()
```

Examiners report

Many students constructed the correct algorithm in pseudocode. Some students used Java or Python code to answer this question and scored full marks. Some used recursion technique to ask for input repeatedly in case of input not in specified range, instead of using a loop.

In the program, the table is stored as a static two-dimensional array.

Figure 3: Two examples of data

Figure 3a: The two-dimensional array is correctly completed because each of the numbers from 1 to 25 appears only once.

	[0]	[1]	[2]	[3]	[4]
[0]	2	24	1	8	15
[1]	23	5	7	14	16
[2]	4	6	13	20	22
[3]	10	12	19	21	3
[4]	11	18	25	17	9

Figure 3b: The two-dimensional array is not correctly completed because the number 17 is missing and the number 12 appears twice.

	[0]	[1]	[2]	[3]	[4]
[0]	12	24	1	8	15
[1]	23	5	7	14	16
[2]	4	6	13	20	22
[3]	10	12	19	21	3
[4]	11	18	25	2	9

The sub-program `isCorrectlyCompleted(MAT)` accepts the two-dimensional array `MAT` and outputs an appropriate message to indicate if it has been correctly filled.

For example:

If `MAT` is the two-dimensional array shown in **Figure 3a**, then `isCorrectlyCompleted(MAT)` should output that the table has been correctly completed.

If `MAT` is the two-dimensional array shown in **Figure 3b** then `isCorrectlyCompleted(MAT)` should output that the table has **not** been

correctly completed.

The description of the algorithm for the `isCorrectlyCompleted(MAT)` sub-program is as follows:

- Initialize a one-dimensional array, `FLAGS`, with zero (0) values.
- Visit all elements of the `MAT` array.
 - Use the number stored in the `MAT` array to match up the index in the `FLAGS` array.
 - Assign one (1) to the `FLAGS` array at that index.
- Inspect elements of the `FLAGS` array.
 - If all the elements in the `FLAGS` array are one (1), output a message saying that the table has been completed correctly; otherwise, output a message saying that the table has not been completed correctly.

For example, the `FLAGS` array for the `MAT` array shown in **Figure 3a** is:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]	[19]	[20]	[21]	[22]	[23]	[24]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

And the `FLAGS` array for the `MAT` array shown in **Figure 3b** is:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]	[19]	[20]	[21]	[22]	[23]	[24]
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1

- (c) Construct the algorithm in pseudocode for the `isCorrectlyCompleted(MAT)` sub-program as described.

[8]

Markscheme
<i>Award max [8]</i> <i>Example 1:</i> <i>Award max [2] for initializing the <code>FLAGS</code> array. Award [1] for correct loop and [1] for correct assignment statement.</i>

Award **max** [4] for matching up the values in the **MAT** array with indexes in the **FLAGS** array and assigning values to elements of the **FLAGS** array. Award [1] for correct row loop, [1] for correct column loop, [1] for calculating index, [1] for assignment statement.

Award **max** [4] for inspecting all elements in the **FLAGS** array and displaying appropriate messages.

Award [1] for use of flag, [1] for correct loop through the array **FLAGS**, [1] for correct condition, [1] for correct flag test and appropriate output (after the loop)

Note: The subprogram heading and return statement may not appear.

isCorrectlyCompleted (MAT)

```
// the FLAGS array holds 25 zeros indexed from 0 to 24
// initializing the FLAGS array
loop K from 0 to 24
    FLAGS[K] = 0
end loop
```

```
// matching up the values in the MAT array with
//indexes in the FLAGS array
// and assigning values to elements of the FLAGS array
loop R from 0 to 4 // 4 or MAT.length-1
    loop C from 0 to 4
        INDEX = MAT[R][C]-1
        FLAGS [INDEX] = 1
    end loop
end loop
```

```
//inspecting all elements in the FLAGS array
//and displaying appropriate messages
```

```
F = true //searching for 0s in the FLAG array
loop K from 0 to 24 // 24 or FLAGS.length-1
    if FLAGS [K] = 0
        then F = false
    end if
end loop
```

```
if F // F==true
    then
        output('the table has been correctly completed ')
```

```

        else
            output('the table has not been correctly completed ')
        end if
        return
    end isCorrectlyCompleted

```

Alternative pseudocode for 'initializing the *FLAGS* array':

Award [2] for

```

FLAGS = [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]

```

Alternative pseudocode for the part "inspecting all elements in the *FLAGS* array and displaying appropriate messages"

```

k=0
loop while k <=24 DO
    if FLAGS[k] == 0
        then output('the table has not been completed correctly')
        return
    endif
    k = k+1
endloop
output('the table has been completed correctly')

```

Example 2:

*Award **max** [8]*

*Award **max** [2] for initializing the *FLAGS* array. Award [1] for correct loop and [1] for correct assignment statement.*

*Award **max** [4] for matching up the values in the *MAT* array with indexes in the *FLAGS* array and assigning values to elements of the *FLAGS* array. Award [1] for correct row loop, [1] for correct column loop, [1] for calculating index, [1] for assignment statement.*

*Award **max** [4] for inspecting all elements in the **FLAGS** array and displaying appropriate messages.*
*Award [1] for initializing counter, [1] for correct loop through the array **FLAGS**, [1] for increasing counter (if needed), [1] for correct test and appropriate output (after the loop)*

*// the **FLAGS** array indexed from 0 to 24*

*// initializing the **FLAGS** array*

loop K from 0 to 24

FLAGS [K] = 0

end loop

*// matching up the values in the **MAT** array with*

*//indexes in the **FLAGS** array*

*// and assigning values to elements of the **FLAGS** array*

loop R from 0 to 4

 loop C from 0 to 4

FLAGS [**MAT**[R][C]–1] = 1

 end loop

end loop

*//inspecting all elements in the **FLAGS** array (counting 1s)*

//and displaying appropriate messages

COUNTER = 0 *//counting 1s in the **FLAG** array*

loop I from 0 to 24

 if **FLAGS** [I]== 1

 then **COUNTER** = **COUNTER** + 1

 end if

end loop

if **COUNTER** == 25 *// all 1s in the **FLAGS** array*

 then output('the table has been completed correctly ')

 else output('the table has not been completed correctly ')

end if

Example 3 (**alternative**):

Award **max** [8].

Award **max** [2] for initializing the **FLAGS** array: [1] for correct loop and [1] for assignment statement.

Award **max** [6] for matching up the values in the **MAT** array with indexes in the **FLAGS** array, assigning values to elements of the **FLAGS** array and for terminating the algorithm early:

[1] for correct row loop

[1] for correct column loop

[1] for calculating correctly index in the **FLAGS** array (index is **MAT[R][C]-1**)

[1] for checking if the value of the **FLAGS** element at that index is 1

[1] for outputting that the table has not been filled in correctly (within the inner loop)

[1] for terminating early the execution of the algorithm (**return** within the inner loop)

[1] for assignment **FLAGS [MAT[R][C]-1] = 1** (if the **FLAGS** element at that index is 0)

[1] for output after the nested loops (outputting that the table has been filled in correctly)

// the FLAGS array holds 25 values indexed from 0 to 24

// initializing the FLAGS array

loop K from 0 to 24

FLAGS [K] = 0

end loop

// matching up the values in the MAT array with

// indexes in the FLAGS array

// and assigning values to elements of the FLAGS array

// this algorithm outputs message and terminates immediately

// in case that a number (MAT[R][C]-1) appears for the second

// time in the FLAGS array (when FLAGS [MAT[R][C]-1] == 1)

loop R from 0 to 4

loop C from 0 to 4

if FLAGS [MAT[R][C]-1] == 1

then

output('the table has not been completed correctly ')

return

```

        else
            FLAGS [MAT[R][C]-1] = 1
        endif
    end loop
end loop

// MAT completed correctly, displaying an appropriate message

output('the table has been completed correctly ')

```

Examiners report

Most students constructed the correct algorithm in pseudocode. Some students made mistakes in the calculation of index and the upper limit of the loop counter. Required algorithm was also presented in Java or Python code which scored full marks.

4. [Maximum mark: 4]

24N.1.SL.TZ0.9

Construct a trace table for the following algorithm:

```

A = 20
B = 12
loop while B > 0
    TEMP = B
    B = A mod B
    A = TEMP
end loop
output (A)

```

[4]

Markscheme

*Award **[4 max]***

*Award **[1]** for each correct column.*

The trace table may be differently presented.

A	B	TEMP	output
20	12	12	
12	8	8	
8	4	4	
4	0		4

Examiners report

Most of the students were able to achieve at least two marks out of four here. Quite a few did not add the variable 'temp' to their trace table and lost a mark.

5. [Maximum mark: 2]

24N.1.SL.TZ0.10

State **two** characteristics of a collection.

[2]

Markscheme

Award [2 max]

Collection holds objects/data that can be of different types/ has different ways of organizing data /objects it contains;
 Collection can hold an unlimited number of values (limited only by the amount of available memory);
 Collection has a set of in-built methods (getNext etc.) that define operations performed on the elements/objects;
 Reduced programming effort (implementations of data structures are provided in collections);
 Increased performance of the program (efficient pre-defined algorithms);

Examiners report

While there were a few good answers to this question, many students need to improve their understanding of what is meant by a collection.

6. [Maximum mark: 15]

24N.1.SL.TZ0.13

Mean and median are different measures of central tendency for a numerical dataset.

Figure 1: Example data values stored in a one-dimensional array, **DATA, which contains 10 elements**

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
2	3	3	6	5	9	10	100	14	17

The mean of the data values stored in a one-dimensional array can be determined as follows:

- Add up all the numbers in the one-dimensional array.
- Divide this sum by how many numbers there are.

The mean of the data values stored in the **DATA** array in **Figure 1** is 16.9.

- (a) Construct an algorithm in pseudocode to calculate and output the mean of the data values stored in the one-dimensional array **DATA** that contains **N** elements. You may assume that the **DATA** array containing **N** elements has been inputted.

[4]

Markscheme

Award [4 max]

*Award [1] for initializing the **MEAN** (or any variable used for summing)*

Award [1] for correct loop including correct use of counter, if any (for loop/while loop/ loop until)

*Award [1] for correctly increasing **MEAN** by **DATA[K]**(correct use of array subscript)*
*Award [1] for calculating and outputting the **MEAN***

Example 1:

```
MEAN=0
loop K from 0 to N-1    //9 or DATA.length-1
    MEAN= MEAN + DATA[K]
end loop
output (MEAN / N)      // MEAN/10 or DATA.length
```

Example 2:

```
M = 0
K = 0
loop while K <= N-1    //9 or DATA.length-1
    M = M + DATA[K]
    K = K + 1
end loop
MEAN = M / N
Output MEAN
```

Examiners report

A range of performance was seen here with some scoring full marks too. The most common mistake was the use of collection methods instead of array subscript to access the elements of the array. Some also lost marks for not initializing the 'Sum' variable or outputting the 'Mean'.

The median of the data values stored in a one-dimensional array can be determined as follows:

- Arrange the data values from the lowest to highest value.
- The median is the data value in the middle of the array.

- If there are two data values in the middle, then the median is the mean of those two values.

The median of the data values stored in the **DATA** array in **Figure 1** is 7.5.

- (b.i) State **one** algorithm that could be used to arrange the data values from the lowest to highest value.

[1]

Markscheme

Award [1 max]

Sorting algorithm;
Bubble sort;
Selection sort;
Insertion sort;
Shell sort;
Merge sort;
Quick sort;
Heap sort;

Examiners report

Mostly well attempted with majority of the students gaining a mark here.

- (b.ii) Construct an algorithm in pseudocode to determine and output the median of the data values stored in the one-dimensional array **DATA** that contains **N** elements. You may assume that the **DATA** array containing **N** elements has been inputted.

[10]

Markscheme

Award [10 max]

Example 1 (Selection sort used to arrange the data values from lowest to highest value):

Award [1] for correct outer loop

Award [1] for correct inner loop

Award [1] for Min initialized to R (within outer loop and before inner loop)

Award [1] for comparing DATA [C] with DATA [Min] and changing the value of Min (if needed)

Award [1] for comparing Min with R (after inner loop)

Award [1] for correct swap

Award [1] for checking whether N is an odd or an even number

Award [1] for calculating correctly MEDIAN (when N is an even number- two data values in the middle of array)

Award [1] for calculating correctly MEDIAN (when N is an odd number- one data value in the middle of array)

Award [1] for outputting MEDIAN

//arranges data values from lowest to highest value

loop R from 0 to N-1

 Min= R

 loop C from R+1 to N-1

 if DATA[C] < DATA[Min]

 then

 Min=C

 end if

 end loop

 if Min != R

 then

 T = DATA[R]

 DATA[R] = DATA[Min]

 DATA[Min] = T

 end if

end loop

//calculates and outputs MEDIAN

if N mod 2 = 0


```

then // two data values in the middle
    //so the median is the mean of these two values
    MEDIAN = (DATA[N div 2]+ DATA[N div 2-1])/2
else
    // median is the data value in the middle of the array
    MEDIAN = DATA[N div 2]
endif
output MEDIAN

```

Example 2 *(Bubble sort logic used to arrange the data values from lowest to highest value)*

Award [1] for correct outer loop (either with a Flag or N-1 times)

Award [1] for correct inner loop (until N-2)

Award [1] for efficient outer loop (use of Flag, values changed correctly in all three places)

Award [1] for efficient inner loop (use of count or similar)

Award [1] for comparing DATA [I] with DATA [I+ 1]

Award [1] for correct swap

Award [1] for checking whether N is an odd or an even number

Award [1] for calculating correctly MEDIAN when N is an even number- two data values in the middle of array

Award [1] for calculating correctly MEDIAN when N is an odd number- one data value in the middle of array

Award [1] for outputting MEDIAN

Sample

//arranges data values from lowest to highest value

count =0

Flag=True

loop while Flag

 Flag=False

 loop I from 0 to N-2-count

 if DATA[I] > DATA[I + 1] then

 TEMP=DATA[I]

 DATA[I] = DATA[I + 1]

```

        DATA[I+1] = TEMP
        Flag=True
    end if
end loop
count = count +1
end loop
//calculates and outputs MEDIAN
X = N mod 2
IND = N div 2
if X = 1
    then // median is the data value in the middle of array
        MEDIAN = DATA[IND]
    else // two data values in the middle
        //so the median is the mean of the two values
        MEDIAN = (DATA[IND]+ DATA[IND-1])/2

endif
output MEDIAN

```

Note: Selection sort and bubble sort are on the syllabus. Other sorting algorithms are acceptable. In case that any other sorting algorithm appears in a candidate's response, contact your team leader for advice.

Examiners report

A small number of good solutions were seen. Most students made little or no viable attempt on this question. Those who did attempt used either a bubble sort or a selection sort. Some students who calculated the median incorrectly was mostly due to ignoring the fact that the first element of the array started at 0 as given in the diagram on the question. An inefficient bubble sort also resulted in a loss of marks.

7. [Maximum mark: 2]

24N.1.HL.TZ0.6

State **two** benefits of using subprograms within a computer program.

[2]

Markscheme

Award [2 max]

Modularity;
Extensibility;
Reusability;
Improved testing/ debugging;
Maintainability;
Abstraction;

Note: A single word does not have to be used, accept descriptions.

Examiners report

Many students attempted this question, and most of the students who attempted, gained full marks. A very few students had difficulty in getting the full marks.

8. [Maximum mark: 15]

24M.1.SL.TZ1.15

(a) Describe **one** standard operation of collections.

[2]

Markscheme

Award [2 max]

`addItem();`

adds an element to the collection (at the end);

`getNext();`

returns / gets / obtains the next element from the collection (indicated by the pointer);

`resetNext();`

restarts the iteration through the collection from the first element / moves the pointer to the beginning of the collection;

`hasNext();`

returns TRUE if there is at least one more element in the collection that has not been accessed (in the current iteration) / if there is any element remaining to access;

`isEmpty();`

returns TRUE if the collection does not contain any elements;

*Note: Award **[1]** mark for name and Award **[1]** mark for the description of collection methods.*

Accept other equivalent method names such as `add()`, `getData()` etc.

Alternative mark points based on syllabus content:

*Award **[1 max]***

Data can be added to the collection (at the current location or at the start);

Data cannot be inserted at a specific location in the collection;

Data can be removed/retrieved from the collection (at the current location or at the start);

Examiners report

Many students had difficulty in getting the full marks. The students did not write the associated description for the operation or method (Note that the description should include the parameters / return value). Some wrote the purpose or application of the method. It is recommended that the students go through the additional document "Pseudocode in Examinations" which is part of the IB resources.

The collection **RAINBOW** holds the names of the main colours of the rainbow along with their order indicator, as follows:

Blue, E, Green, D, Indigo, F, Orange, B, Red, A, Violet, G, Yellow, C

- (b) Construct an algorithm using pseudocode to read the data from **RAINBOW** and store the names of the colours in a one-dimensional string array, **COLOUR**, and the order indicators in another one-dimensional string array, **ORDER**.

The matching colour and order indicator must have the same index in the **COLOUR** and **ORDER** arrays.

[4]

Markscheme

Award [1 max]

Correct use of collection methods `hasNext()`, `getNext()` (, `resetNext()`) or equivalent methods;

Award [3 max]

Initializing the starting array index as 0;
Appropriate loop for accessing **RAINBOW** elements;

Correct assignment to both arrays using the same index, with two adjacent elements from RAINBOW;
Array indexes increment for whole data set;

Example answer

```
RAINBOW.resetNext()
I = 0
loop while RAINBOW.hasNext()
  COLOUR[I] = RAINBOW.getNext()
  ORDER[I] = RAINBOW.getNext()
  I = I + 1
end loop
```

Examiners report

Some good algorithms were seen for this question. Few students had mixed up the operations associated with a collection and an array. They had used collection methods for array / index for collection.

- (c) Construct an algorithm using pseudocode to sort the contents of the array **ORDER** into alphabetical order. The indexes for the corresponding data in the two parallel arrays must remain the same after sorting.

[5]

Markscheme

Award [5 max]

Bubble Sort

Use of nested loops with correct conditions / initial and final values;
Use of flag to stop the loop if no swaps made;

Comparing adjacent elements of **ORDER** array;
Swapping contents in **ORDER** array if required;
Corresponding swapping of contents in **COLOUR** array if required;

Note: Do not accept break statement.

Example 1

```
SWAP = TRUE
loop while SWAP = TRUE
  SWAP = FALSE
  loop I from 0 to 5
    if ORDER[I] > ORDER[I + 1] then
      TEMP1 = ORDER[I]
      TEMP2 = COLOUR[I]
      ORDER[I] = ORDER[I + 1]
      COLOUR[I] = COLOUR[I + 1]
      ORDER[I + 1] = TEMP1
      COLOUR[I + 1] = TEMP2
      SWAP = TRUE
    end if
  end loop
end loop
```

Bubble Sort

Use of nested loops;
Looping with the correct initial and final values;
Comparing adjacent elements of **ORDER** array;
Swapping contents in **ORDER** array if required;
Corresponding swapping of contents in **COLOUR** array if required;

Example 2

```

loop J from 0 to 5 // Accept loop J from 0 to 6
  loop I from 0 to 6-J-1
    if ORDER[I] > ORDER[I + 1] then
      TEMP1 = ORDER[I]
      TEMP2 = COLOUR[I]
      ORDER[I] = ORDER[I + 1]
      COLOUR[I] = COLOUR[I + 1]
      ORDER[I + 1] = TEMP1
      COLOUR[I + 1] = TEMP2
    end if
  end loop
end loop

```

Selection Sort

Use of nested loops with correct initial and final values;
 Setting of **MIN** using correct index before the inner loop;
 Comparing elements in **ORDER** array to find smallest value inside the correct loop;
 Swapping contents in **ORDER** array inside the correct loop;
 Corresponding swapping of contents in **COLOUR** array;

Example 3

```

loop I from 0 to 5 // Accept loop I from 0 to 6
  MIN = I
  loop J from I + 1 to 6
    if ORDER[J] < ORDER[MIN] then
      MIN = J
    end if
  end loop
  if min <> I then
    TEMP1 = ORDER[MIN]
    TEMP2 = COLOUR[MIN]
    ORDER[MIN] = ORDER[I]
    COLOUR[MIN] = COLOUR[I]
    ORDER[I] = TEMP1
    COLOUR[I] = TEMP2
  end if
end loop

```



```
end if  
end loop
```

Selection Sort

Use of nested loops with correct initial and final values;
Setting of MIN value and correct initialization of POS;
Comparing elements in ORDER array to find smallest value and storing the POS;
Swapping contents in ORDER array inside the correct loop;
Corresponding swapping of contents in COLOUR array;

Example 4

```
loop I from 0 to 5 // Accept loop I from 0 to 6  
  MIN = ORDER[I}  
  POS = I  
  loop J from I + 1 to 6  
    if ORDER[J] < MIN then  
      MIN = ORDER[J}  
      POS = J  
    end if  
  end loop  
  if POS <> I then  
    TEMP1 = ORDER[MIN]  
    TEMP2 = COLOUR[MIN]  
    ORDER[MIN] = ORDER[I]  
    COLOUR[MIN] = COLOUR[I]  
    ORDER[I] = TEMP1  
    COLOUR[I] = TEMP2  
  end if  
end loop
```

Examiners report

Some students were confident in producing a bubble sort or a selection sort algorithm for this question. Majority of the students made mistakes in defining the loop limits, comparison of adjacent elements / finding the

smallest element and exchanging the required elements. Some had combined the code for bubble sort and selection sort.

- (d) Compare and contrast the bubble sort algorithm and the selection sort algorithm.

[4]

Markscheme

Award [4 max]

Note: At least one similarity and one difference to be mentioned

Similarities:

Both use nested loops, (each time reducing the inner loop);

Both have the same worst case time complexity ($O(n^2)$);

Both perform in place sorting directly within the array (without requiring additional data structures / memory);

Differences:

Bubble sort works by comparing adjacent elements, swapping the elements if they are in wrong order, (repeating these steps until the array is fully sorted);

Selection sort works by selecting the smallest (or largest) element from the unsorted portion of the array and swapping it with the first unsorted element;

Bubble sort can exit early/ is faster if the list is already sorted;

Selection sort will need to complete the procedure for the entire list every time;

Bubble sort has a higher number of comparisons and swaps, making it slower compared to selection sort for larger datasets;

Bubble sort has a best-case time complexity $O(n)$ whereas Selection sort has a best-case time complexity $O(n^2)$;

Examiners report

Students who wrote at least one similarity and listed the differences achieved the highest marks. It is not sufficient to say "Both are sorting algorithms" for similarity, students should cite correct technical points for similarities (Both use nested loops, have same time complexities) and differences.

9. [Maximum mark: 15]

24M.1.SL.TZ2.15

(a) Describe **one** standard operation of collections.

[2]

Markscheme

Award [2 max]

`addItem();`

adds an element to the collection (at the end);

`getNext();`

returns / gets / obtains the next element from the collection (indicated by the pointer);

`resetNext();`

restarts the iteration through the collection from the first element / moves the pointer to the beginning of the collection;

`hasNext();`

returns TRUE if there is at least one more element in the collection that has not been accessed (in the current iteration) / if there is any element remaining to access;

`isEmpty();`

returns TRUE if the collection does not contain any elements;

*Note: Award [1] mark for name and Award [1] mark for the description of collection methods.
Accept other equivalent method names such as add(), getData() etc.*

*Alternative mark points based on syllabus content:
Award [1 max];*

Data can be added to the collection (at the current location or at the start);
Data cannot be inserted at a specific location in the collection;
Data can be removed/retrieved from the collection (at the current location or at the start);

Examiners report

Many students had difficulty in getting the full marks. The students did not write the associated description for the operation or method (Note that the description should include the parameters / return value). Some wrote the purpose or application of the method. It is recommended that the students go through the additional document "Pseudocode in Examinations" which is part of the IB resources.

The collection CAPITALS holds the names of a number of capital cities and the names of their corresponding countries, as follows:

Ankara, Turkey, Brasilia, Brazil, Dhaka, Bangladesh, Lisbon,
Portugal, Manila, Philippines, Rome, Italy

- (b) Construct an algorithm using pseudocode to read the data from CAPITALS and store the names of the cities in a one-dimensional string array, CITY, and the names of the countries in another one-dimensional string array, COUNTRY.

The matching city and country must have the same index in the CITY and COUNTRY arrays.

[4]

Markscheme

Award [1 max]

Correct use of collection methods hasNext(), getNext() (, resetNext()) or equivalent methods;

Award [3 max]

Initializing the array starting index as 0;

Appropriate loop for accessing CAPITALS elements;

Correct assignment to both arrays using the same index, with two adjacent elements from CAPITALS;

Array indexes increment for whole data set;

Example answer

```
CAPITALS.resetNext()
```

```
I = 0
```

```
loop while CAPITALS.hasNext()
```

```
    CITY[I] = CAPITALS.getNext()
```

```
    COUNTRY[I] = CAPITALS.getNext()
```

```
    I = I + 1
```

```
end loop
```

Examiners report

Some good algorithms were seen for this question. Few students had mixed up the operations associated with a collection and an array. They had used

collection methods for array / index for collection.

- (c) Construct an algorithm using pseudocode to sort the contents of the array **COUNTRY** into alphabetical order. The indexes for the corresponding data in the two parallel arrays must remain the same after sorting.

[5]

Markscheme

Award [5 max]

Bubble Sort

Use of nested loops with correct conditions / initial and final values;

Use of flag to stop the loop if no swaps made;

Comparing adjacent elements of **COUNTRY** array;

Swapping contents in **COUNTRY** array if required;

Corresponding swapping of contents in **CITY** array if required;

Example 1

SWAP = TRUE

loop while SWAP = TRUE

 SWAP = FALSE

 loop I from 0 to 4

 if **COUNTRY**[I] > **COUNTRY**[I + 1] then

 TEMP1 = **COUNTRY**[I]

 TEMP2 = **CITY**[I]

COUNTRY[I] = **COUNTRY**[I + 1]

CITY[I] = **CITY**[I + 1]

COUNTRY[I + 1] = TEMP1

CITY[I + 1] = TEMP2

 SWAP = TRUE

 end if

```
    end loop  
end loop
```

Bubble Sort

Use of nested loops;

Looping with the correct initial and final values;

Comparing adjacent elements of COUNTRY array;

Swapping contents in COUNTRY array if required;

Corresponding swapping of contents in CITY array if required;

Example 2

```
loop J from 0 to 4 // Accept loop J from 0 to 5  
  loop I from 0 to 5-J-1  
    if COUNTRY[I] > COUNTRY[I + 1] then  
      TEMP1 = COUNTRY[I]  
      TEMP2 = CITY[I]  
      COUNTRY[I] = COUNTRY[I + 1]  
      CITY[I] = CITY[I + 1]  
      COUNTRY[I + 1] = TEMP1  
      CITY[I + 1] = TEMP2  
    end if  
  end loop  
end loop
```

Selection Sort

Use of nested loops with correct initial and final values;

Setting of MIN using correct index before the inner loop;

Comparing elements in COUNTRY array to find the smallest value inside the correct loop;

Swapping contents in COUNTRY array inside the correct loop;

Corresponding swapping of contents in CITY array;

Example 3

```
loop I from 0 to 4 // Accept loop J from 0 to 5  
  MIN = I  
  loop J from I + 1 to 5
```

```

    if COUNTRY[J] < COUNTRY[MIN] then
        MIN = J
    end if
end loop
if min <> I then
    TEMP1 = COUNTRY[MIN]
    TEMP2 = CITY[MIN]
    COUNTRY[MIN] = COUNTRY[I]
    CITY[MIN] = CITY[I]
    COUNTRY[I] = TEMP1
    CITY[I] = TEMP2
end if
end loop

```

Selection Sort

Use of nested loops with correct initial and final values;

Setting of **MIN** value and correct initialization of **POS**;

Comparing elements in **COUNTRY** array to find the smallest value and changing the **POS**;

Swapping contents in **COUNTRY** array inside the correct loop;

Corresponding swapping of contents in **CITY** array;

Example 4

loop I from 0 to 4 // Accept loop J from 0 to 5

```

    MIN = COUNTRY[I]

```

```

    POS = I

```

```

    loop J from I + 1 to 5

```

```

        if COUNTRY[J] < MIN then

```

```

            MIN = COUNTRY[J]

```

```

            POS = J

```

```

        end if

```

```

    end loop

```

```

    if POS <> I then

```

```

        TEMP1 = COUNTRY[MIN]

```

```

        TEMP2 = CITY[MIN]

```

```

        COUNTRY[MIN] = COUNTRY[I]

```



```
CITY[MIN] = CITY[I]
COUNTRY[I] = TEMP1
CITY[I] = TEMP2
end if
end loop
```

Examiners report

Some students were confident in producing a bubble sort or a selection sort algorithm for this question. Majority of the students made mistakes in defining the loop limits, comparison of adjacent elements / finding the smallest element and exchanging the required elements. Some had combined the code for bubble sort and selection sort.

- (d) Compare and contrast the bubble sort algorithm and the selection sort algorithm.

[4]

Markscheme

Award [4 max]

Note: At least one similarity and one difference to be mentioned

Similarities:

Both use nested loops, (each time reducing the inner loop);
Both have the same worst case time complexity ($O(n^2)$);
Both perform in place sorting directly within the array (without requiring additional data structures / memory);

Differences:

Bubble sort works by comparing adjacent elements, swapping the elements if they are in wrong order, (repeating these steps until the array is

fully sorted);

Selection sort works by selecting the smallest (or largest) element from the unsorted portion of the array and swapping it with the first unsorted element;

Bubble sort can exit early/ is faster if the list is already sorted;

Selection sort will need to complete the procedure for the entire list every time;

Bubble sort has a higher number of comparisons and swaps, making it slower compared to selection sort for larger datasets;

Bubble sort has a best-case time complexity $O(n)$ whereas Selection sort has a best-case time complexity $O(n^2)$;

Examiners report

Students who wrote at least one similarity and listed the differences achieved the highest marks. It is not sufficient to say "Both are sorting algorithms" for similarity, students should cite correct technical points for similarities (Both use nested loops, have same time complexities, etc.) and differences.

10. [Maximum mark: 1]
Define the term *constant*.

23N.1.SL.TZ0.2

[1]

Markscheme

Award [1 max].

Constant is a value of data which cannot be altered during execution of a program;

11. [Maximum mark: 3]

23N.1.SL.TZ0.8

(a) Outline what is meant by concurrent processing.

[2]

Markscheme

Award [2 max].

Concurrent processing means that a single or multiple system perform several tasks; simultaneously/within overlapping time frames;

Concurrent processing means execution of several algorithms/programs/sub-programs;

At the same time (regardless of the number of processors);

Examiners report

SL:

There were a small number of completely correct answers to this question with many students getting at least one mark for the mention of "simultaneous/at the same time".

HL:

These two questions (a and b) highlight weakness in knowledge of the basic theory, as many students seemingly did not recognise the term 'concurrent processing'.

(b) Identify **one** advantage of concurrent processing.

[1]

Markscheme

Award [1 max].

Better use of (computer) resources;

Decreased response time/waiting time/increased efficiency;

Reduced overall run-time of the program;

More real-world problems can be solved;

Examiners report

Students who gave a more specific answer identifying the advantage of concurrent processing, scored more marks than those who gave one word answer such as 'fast' and 'quick'.

12. [Maximum mark: 6]

23N.1.SL.TZ0.9

(a) Construct a trace table for the following algorithm:

```
N = 1216
X = 0
loop while N > 0
    X = X + N mod 10
    N = N div 10
end loop
output (X)
```

[4]

Markscheme

Award [4 max].

Award [1] for a trace table with at least 2 columns (N, X, output)

Award [1] for the correct column N

Award [1] for the correct column X

Award [1] for the correct output

N	X	N>0	output
1216	0	True	
121	6	True	
12	7	True	
1	9	True	
0	10	False	10

Note: The trace table may be presented differently.

Alternate Mark Scheme:

Award [4 max].

Award [1] for a trace table with at least 2 columns (N, X)
Award [1] for the first two rows (with correct values of N and X)
Award [1] for the next two rows (with correct values of N and X)
Award [1] for the last row (with correct values of N, X and output)

Examiners report

Almost all students scored at least one mark for making a table with column N and X. A few students did not realize that $N \div 10$ results in an integer value that gives the wrong output.

(b) Deduce the purpose of the algorithm.

[2]

Markscheme

Award [2 max].
 Calculates/outputs the sum;
 Of all digits in N;

Examiners report

Students who lost marks in part (a) found it difficult to figure out the purpose of the algorithm and those who did, failed to specify that it adds up 'the individual digits of N'.